

Relazione del progetto d'esame di Editoria Digitale

Matteo Baraggini

a.a. 2025/2026

Scienza sotto pressione

Newsletter di giornalismo scientifico

Appello del 9 luglio 2026

Indice

1	Introduzione	2
2	Ideazione	2
2.1	Tema	2
2.2	Destinatari	2
3	Requisiti di accettazione	3
3.1	Canali di distribuzione	4
4	Processo di Produzione	4
4.1	Acquisizione dei contenuti	4
4.2	Gestione documentale	5
4.3	Tecnologie adottate	6
4.4	Esecuzione del flusso	7
4.5	Utilizzo di intelligenza artificiale generativa	8
5	Valutazione dei risultati raggiunti	10
5.1	Valutazione del flusso di produzione	10
5.2	Confronto AS-IS / TO-BE	11
5.3	Limiti emersi	11
6	Conclusioni	11
	Bibliografia e sitografia	12

1 Introduzione

Il presente progetto consiste nello sviluppo di *Scienza sotto pressione*, una newsletter scientifica digitale rivolta a giornalisti, operatori dell'informazione e comunicatori della conoscenza. Il prodotto fornisce aggiornamenti rapidi, affidabili e accessibili su temi scientifici di attualità, basandosi esclusivamente su articoli pubblicati con licenza open access.

I temi trattati nel numero prototipale sono l'impatto dell'**intelligenza artificiale in sanità** e il fenomeno della **disinformazione online in ambito medico** — due argomenti strettamente interconnessi e di grande rilevanza per il giornalismo scientifico contemporaneo.

Dal punto di vista tecnologico, il flusso di produzione adotta **Markdown** come formato sorgente, **Pandoc** come strumento di conversione multiformato e **WeasyPrint** per la generazione del PDF: a differenza di un motore LaTeX, WeasyPrint converte in PDF l'HTML già prodotto da Pandoc applicando lo stesso foglio di stile CSS del progetto, così che l'identità editoriale sia identica su entrambi i formati di output (si veda la Sezione 4.4 per il dettaglio). La ricerca delle fonti e la generazione degli elementi visivi sono supportate da uno strato di **automazione in Python**: uno script interroga l'indice bibliografico aperto OpenAlex per individuare articoli open access pertinenti, un secondo genera automaticamente i grafici integrati nel numero. La generazione dei contenuti editoriali, e in parte lo sviluppo degli strumenti di automazione, sono stati supportati dall'uso di un Large Language Model (Claude, Anthropic) — si veda la Sezione 4.5 per il dettaglio. Il versioning e la distribuzione dei materiali avvengono tramite repository pubblico su GitHub.

2 Ideazione

2.1 Tema

I temi selezionati per il prodotto editoriale sono due, entrambi centrali nel dibattito pubblico attuale:

- **Intelligenza artificiale in sanità**: l'adozione di sistemi AI in ambito medico è uno dei fenomeni tecnologici più discussi degli ultimi anni. Con l'approvazione dell'AI Act europeo (2024) e la diffusione di strumenti basati su modelli di linguaggio in contesti clinici, il tema è diventato urgente anche per il giornalismo generalista.
- **Disinformazione medica online**: il fenomeno della misinformazione sanitaria sui social media ha raggiunto visibilità globale durante la pandemia di COVID-19, ma ha radici più profonde e un perimetro più ampio. La ricerca scientifica sul tema è cresciuta esponenzialmente negli ultimi cinque anni.

I due temi sono correlati: i sistemi di intelligenza artificiale generativa sono sempre più impiegati sia nella produzione di contenuti informativi affidabili sia, per converso, nella diffusione di disinformazione automatizzata. Questo legame aumenta la rilevanza editoriale del prodotto.

2.2 Destinatari

Il prodotto è rivolto a tre categorie di destinatari, descritte attraverso le seguenti personas:

Persona 1 — Giulia, 34 anni, giornalista scientifica freelance. Giulia collabora con testate online e magazine di divulgazione scientifica. Ha una formazione universitaria in biologia,

ma copre temi trasversali come tecnologia, salute e ambiente. Ha poco tempo per leggere articoli accademici completi e ha bisogno di sintesi affidabili, già verificate nella fonte, che le permettano di costruire rapidamente il contesto per i suoi pezzi. Usa prevalentemente il telefono per aggiornarsi, quindi apprezza contenuti brevi, ben strutturati e con link diretti alle fonti.

Scenario d'uso: Giulia deve scrivere un articolo sull'AI in medicina entro domani. Riceve la newsletter, legge la sintesi dell'articolo di Faiyazuddin et al. (2025), osserva a colpo d'occhio il grafico sull'andamento delle pubblicazioni per contestualizzare il trend, segue il link al testo completo su PMC per verificare i dati, e integra le informazioni nel suo pezzo citando la fonte.

Persona 2 — Marco, 45 anni, responsabile editoriale di un blog istituzionale sanitario. Marco coordina la comunicazione digitale di un'azienda ospedaliera. Ha bisogno di contenuti aggiornati sui rischi e le opportunità dell'AI in sanità da condividere con i colleghi e da usare come spunto per i comunicati stampa. Preferisce formati compatibili con copia-incolla su CMS web e newsletter aziendali.

Scenario d'uso: Marco riceve l'HTML della newsletter — un file singolo e autonomo, senza dipendenze esterne — lo condivide internamente con il team medico-scientifico, e usa l'introduzione alla sezione AI come base per un post sul sito istituzionale.

Persona 3 — Sara, 28 anni, studentessa di comunicazione scientifica. Sara frequenta un master in science communication. Usa la newsletter come strumento di aggiornamento e come esempio di pratiche editoriali digitali. Apprezza la trasparenza sulle fonti e il formato open access.

Scenario d'uso: Sara usa la newsletter come riferimento per un elaborato sul giornalismo scientifico digitale, citando le fonti open access, il flusso di produzione automatizzato e la nota sul metodo pubblicata in coda al numero.

3 Requisiti di accettazione

I requisiti di accettazione del prodotto si articolano in due dimensioni, seguendo il modello TAM (Technology Acceptance Model, Davis 1989):

Utilità percepita:

- Qualità e verificabilità dei contenuti: ogni sintesi è basata su articoli peer-reviewed con licenza open access e link diretto a PMC
- Aggiornamento continuo: il formato Markdown consente di aggiornare e ridistribuire il prodotto con costi minimi; la ricerca delle fonti è automatizzata e ripetibile su nuovi argomenti senza intervento manuale
- Rilevanza per il target: i temi selezionati sono tra i più discussi nel giornalismo scientifico attuale
- Comprensione immediata dei numeri chiave grazie ai due grafici in apertura, pensati per chi ha poco tempo

Facilità d'uso:

- Formato HTML autonomo (*self-contained*): stile e immagini incorporati nel file, leggibile su qualsiasi browser senza connessione a risorse esterne
- Formato PDF compatibile con la stampa e la distribuzione via email

- Struttura con indice dei contenuti per navigazione rapida
- Linguaggio accessibile ma rigoroso, calibrato sul target professionale
- Pipeline eseguibile con un solo comando (`./build.sh`), anche in assenza di connessione di rete grazie alla modalità `--offline`

3.1 Canali di distribuzione

Canale	Formato	Note
Newsletter (Substack, Mailchimp)	HTML	Copia diretta del file HTML, senza dipendenze esterne (font di sistema)
Blog / CMS Distribuzione via email	HTML PDF	Pubblicazione via upload o copia Output WeasyPrint (stesso CSS dell'HTML)
Social media (link)	HTML	URL al repository GitHub Pages
Repository documentale	MD + YAML + PY	Sorgenti, script di automazione e dati su GitHub

Identità visiva: lo stile tipografico adotta un'impostazione editoriale, con font serif per il corpo del testo e sans-serif per i titoli. Il CSS definisce una larghezza massima del contenuto di 700px, tipica delle newsletter professionali, con testo giustificato e una gerarchia visiva marcata: masthead con occhiello, numerazione automatica delle sezioni con accento colore dedicato a ciascun filone tematico, badge per la licenza open access sulle citazioni, capolettera sull'introduzione, riquadri dedicati ai grafici e indice ristilizzato.

4 Processo di Produzione

4.1 Acquisizione dei contenuti

Le fonti utilizzate sono articoli scientifici open access reperiti su **PubMed Central (PMC)**, verificati per licenza Creative Commons. La ricerca è supportata da uno script dedicato, `search_articles.py`, che interroga programmaticamente l'indice bibliografico aperto **OpenAlex** (stessa tipologia di fonti indicata dalla traccia: PMC, Zenodo, editori open science) in due modalità distinte, pensate per momenti diversi del ciclo di vita del progetto:

- **Scoperta** (`--batch data/argomenti.txt`): query testuali in inglese che propongono nuovi candidati su un argomento, filtrati con `is_oa:true` e ordinati per rilevanza testuale rispetto alla query (non per numero di citazioni, per evitare che review molto citate ma generiche scavalchino uno studio specifico e pertinente). È il punto d'ingresso per chi usa il progetto da zero su un nuovo argomento: restituisce già titolo, autori, anno, stato open access e DOI di ogni candidato, dati sufficienti per la selezione editoriale senza bisogno di ulteriori controlli in quel momento.

- **Verifica** (`--verify data/fonti_selezionate.txt`): lookup diretto per DOI, pensato non per la selezione iniziale ma per un momento successivo e disaccoppiato nel tempo. La ricerca per scoperta non è deterministica: l'indice OpenAlex viene aggiornato costantemente, quindi la stessa query può restituire elenchi diversi a distanza di settimane o mesi, e non garantisce che un articolo già scelto ricompaia tra i risultati. `--verify` permette di controllare, mesi dopo la pubblicazione del numero — o quando qualcun altro clona il repository e riesegue la pipeline — che le fonti citate in `articolo.md` esistano ancora e mantengano lo stato open access dichiarato, senza dover ripetere né dipendere dalla ricerca originale. È uno strumento di audit delle citazioni già fatte, non un passaggio necessario nel flusso di selezione di un nuovo numero.

La selezione editoriale finale — quali articoli sintetizzare tra i candidati proposti dalla scoperta — resta una scelta umana, come richiesto esplicitamente dalla traccia d'esame ("dimostrare la capacità di selezionare e strutturare materiali significativi"): l'automazione copre il reperimento delle fonti e, in un secondo momento, la verifica della loro persistenza, non la scrittura editoriale. Gli articoli selezionati per questo numero sono:

Autori	Titolo	Rivista	Anno
Faiyazuddin et al.	The Impact of AI on Health-care	Health Science Reports	2025
Botha et al.	AI Threats to Patient Rights and Safety	Archives of Public Health	2024
Adebesin et al.	Social Media & Health Misinformation	JMIR Infodemiology	2023
Yeung et al.	Medical Misinformation on Social Media	J Med Internet Research	2022

4.2 Gestione documentale

Il flusso di gestione documentale si articola nelle seguenti fasi:

1. **Ricerca automatica di nuovi candidati:** esecuzione di `search_articles.py --batch` su OpenAlex, con filtro open access ed esportazione dei risultati in CSV per argomento
2. **Selezione editoriale:** scelta manuale, tra i candidati, degli articoli più rappresentativi per il target
3. **Redazione assistita da LLM:** generazione, tramite Claude, delle introduzioni giornalistiche e delle sintesi divulgative a partire dagli abstract degli articoli selezionati (si veda la Sezione 4.5 per prompt e metodo)
4. **Revisione:** controllo manuale dei testi generati, verifica dell'accuratezza rispetto agli abstract originali, correzione di correttezza dei dati e adattamento al formato Markdown
5. **Strutturazione del documento:** redazione del file `articolo.md` con metadati separati in `metadati.yaml`, uso di `fenced div` Pandoc per applicare classi di stile compatibili sia con l'output HTML sia con l'output PDF
6. **Generazione automatica dei grafici:** esecuzione di `make_chart.py`, con fallback su dati campione in assenza di rete
7. **Applicazione dello stile grafico:** foglio di stile `stile.css` per l'output HTML

8. **Conversione e produzione:** esecuzione di `build.sh`, che genera grafici e comandi Pandoc per l'output HTML (self-contained) e PDF
9. **Distribuzione:** pubblicazione su repository GitHub con README documentale

A queste fasi si affianca, non in sequenza ma a cadenza periodica successiva alla pubblicazione, la **verifica diretta delle fonti** (`search_articles.py --verify`): un controllo di audit che, a distanza di tempo, conferma che le fonti citate esistano ancora e mantengano lo stato open access dichiarato, indipendentemente dal ranking di ricerca del momento.

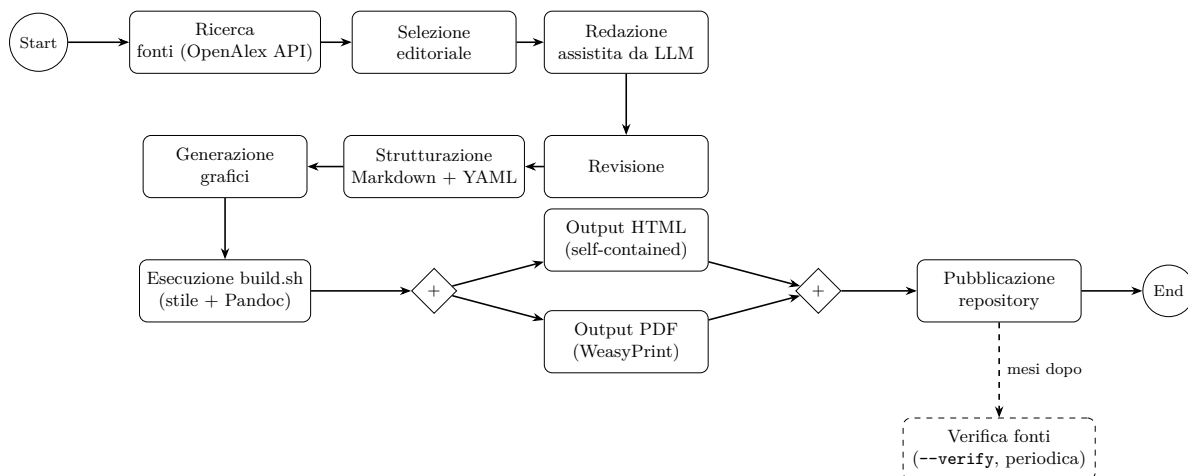


Figura 1: Flusso di produzione documentale. Il riquadro tratteggiato rappresenta un controllo di audit successivo, non un passaggio sequenziale della pipeline.

4.3 Tecnologie adottate

Tecnologia	Fase	Contributo
Markdown	Redazione	Formato sorgente portabile, indipendente da software proprietari, versionabile
YAML	Metadati	Separazione tra contenuto e metadati, interoperabile con Pandoc
Pandoc	Conversione	Produzione di output multipli (HTML, PDF) da unico sorgente, con <i>fenced div</i> e figure implicite per compatibilità cross-formato
CSS	Stile HTML	Identità visiva, separazione tra presentazione e contenuto
WeasyPrint	Output PDF	Rendering CSS-to-PDF dell'HTML generato da Pandoc: applica <code>stile.css</code> anche al PDF, garantendo la stessa identità visiva (colori di sezione, badge open access, capolettera, riquadri grafici) invece di affidarsi al sistema di formattazione separato di LaTeX
Python (requests)	Automazione	Interrogazione dell'API OpenAlex per la ricerca automatica delle fonti
Python (Matplotlib)	Automazione	Generazione automatica dei grafici della newsletter

Tecnologia	Fase	Contributo
Claude (LLM)	Contenuti e automazione	Generazione di introduzioni e sintesi divulgative; sviluppo degli script di automazione e dello stile (§4.5)
GitHub	Versioning	Gestione versioni e repository documentale

4.4 Esecuzione del flusso

I materiali per riprodurre il flusso di produzione sono disponibili nel repository GitHub del progetto. La struttura è la seguente:

```
/
|-- README.md
|-- articolo.md <- contenuto in Markdown
|-- metadati.yaml <- metadati separati (Dublin Core incluso)
|-- stile.css <- foglio di stile HTML
|-- build.sh <- grafici + conversione Pandoc HTML/PDF
|-- scripts/
| |-- cerca.sh <- ricerca + verifica fonti (dipende dalla rete)
| |-- search_articles.py <- ricerca/verifica fonti (OpenAlex API)
| |-- make_chart.py <- generazione grafici
|-- data/
| |-- argomenti.txt <- argomenti per la ricerca batch
| |-- fonti_selezionate.txt <- DOI delle fonti citate, per la verifica diretta
| |-- trend_sample.json <- dati di riserva offline
| |-- risultati_*.csv <- output della ricerca (a runtime)
|-- output/
| |-- articolo.html
| |-- articolo.pdf
| |-- img/*.png <- grafici generati
```

La pipeline è divisa in due fasi indipendenti, ciascuna eseguibile anche da sola, perché hanno caratteristiche diverse: la ricerca dipende dalla rete ed è la parte esplorativa del progetto (ha senso rilanciarla da sola a cadenza periodica, per scoprire nuovi candidati), mentre la generazione del documento è deterministica e deve funzionare sempre, anche offline. Per questo la generazione vive direttamente in `build.sh`, nella radice del progetto, mentre la ricerca è isolata in `scripts/cerca.sh`:

```
# scripts/cerca.sh -- fase di ricerca (rete)
# 1. Ricerca automatica di nuovi candidati (OpenAlex, batch su piu' argomenti)
python3 scripts/search_articles.py --batch data/argomenti.txt \
  --oa-only --top 10 --min-year 2020

# 2. Verifica diretta per DOI delle fonti gia' sintetizzate in articolo.md
python3 scripts/search_articles.py --verify data/fonti_selezionate.txt

# build.sh -- fase di produzione documentale (locale)
# 3. Generazione dei grafici (live o --offline)
python3 scripts/make_chart.py --topic "artificial intelligence AND healthcare"

# 4. Output HTML (self-contained: CSS e immagini incorporati)
pandoc -s articolo.md --metadata-file=metadati.yaml --css=stile.css \
  --toc --toc-depth=2 --highlight-style=kate --self-contained \
  -o output/articolo.html
```



```
# 5. Output PDF (WeasyPrint: applica stile.css anche al PDF)
pandoc articolo.md --metadata-file=metadati.yaml --css=stile.css \
--toc --toc-depth=2 --pdf-engine=weasyprint \
-o output/articolo.pdf
```

L'esecuzione può avvenire in due modi: `./build.sh` (grafici live + generazione HTML/PDF) oppure `./build.sh --offline` (nessuna chiamata di rete, grafici da dati campione). La fase di ricerca si lancia a parte, quando serve, con `./scripts/cerca.sh`.

Motivo della scelta di WeasyPrint come motore per il PDF: nella prima versione della pipeline il PDF veniva generato con `--pdf-engine=xelatex`. XeLaTeX però ignora completamente `stile.css` e applica la propria formattazione LaTeX (margini, font, interlinea impostati via variabili `-V`): l'identità editoriale definita nel CSS — colori di sezione, badge open access, capolettera, riquadri per i grafici, indice ristilizzato — restava quindi visibile solo nell'HTML, mentre il PDF appariva come un documento tipograficamente diverso e più generico. WeasyPrint risolve il problema perché non è un motore LaTeX ma un motore di rendering CSS-to-PDF: prende in input l'HTML già generato da Pandoc e lo trasforma in PDF applicando lo stesso foglio di stile usato per il web, così che i due formati di output condividano esattamente la stessa identità visiva, requisito esplicito emerso in fase di valutazione del prodotto. Il passaggio ha inoltre reso necessarie alcune correzioni nel CSS (si veda la Tabella 4), perché WeasyPrint supporta solo un sottoinsieme delle proprietà CSS3 disponibili nei browser.

4.5 Utilizzo di intelligenza artificiale generativa

Tecnologia adottata: Claude (Anthropic), Large Language Model della famiglia dei Generative Pre-trained Transformer, utilizzato in modalità agentica: accesso diretto ai file di progetto ed esecuzione di script e comandi in un ambiente di lavoro, non solo come generatore di testo da incollare manualmente.

Fasi di integrazione e tipi di elaborazione:

- Generazione di testo narrativo (introduzioni giornalistiche) e sintesi divulgative a partire dagli abstract originali
- Sviluppo di codice: script Python di automazione (ricerca fonti, generazione grafici), foglio di stile CSS, script di build
- Collaudo: esecuzione della pipeline, ispezione visiva dell'output HTML e PDF tramite rendering di pagina, individuazione e correzione di problemi tecnici emersi in fase di esecuzione (Tabella 4)

Approccio di prompt engineering: per la parte testuale sono stati utilizzati 7 prompt, organizzati in due tipologie strutturali.

Tipologia A — Prompt per introduzioni tematiche (3 prompt): struttura con assegnazione di ruolo, definizione del target, contesto temporale, indicazione del contenuto da introdurre. Esempio:

```
Sei un redattore scientifico che scrive per giornalisti
generalisti. Scrivi un'introduzione giornalistica di 2-3
paragrafi per una sezione di newsletter dedicata all'impatto
dell'intelligenza artificiale in ambito sanitario e sociale.
Tono accessibile ma rigoroso, contesto attuale 2025-2026.
Deve introdurre due articoli scientifici: uno sull'impatto
```

dell'AI nella sanità, uno sui rischi per i diritti dei pazienti.

Tipologia B — Template per sintesi divulgative (4 prompt): struttura fissa e riutilizzabile applicata a quattro abstract diversi. I quattro prompt sono strutturalmente identici — scelta consapevole che dimostra la generalizzabilità del template e riduce i costi di produzione per numeri futuri della newsletter.

Sei un redattore scientifico che scrive per giornalisti generalisti. Ti fornisco l'abstract di un articolo scientifico open access. Scrivi una sintesi divulgativa di circa 150 parole che includa: tema dell'articolo, metodo usato, risultati principali, rilevanza pratica per il pubblico. Linguaggio accessibile ma preciso. Concludi con una riga che indica rivista, anno e link alla fonte.

[ABSTRACT: ...]

Per la parte di automazione, le istruzioni sono state espresse come requisiti funzionali (ad esempio: “la ricerca deve filtrare per licenza open access e ordinare per rilevanza”) lasciando al modello la scelta dell’implementazione, verificata poi tramite esecuzione diretta.

La tabella seguente riporta alcuni problemi tecnici reali individuati durante il collaudo della pipeline, a dimostrazione della validazione sistematica applicata a ogni output prima dell’inclusione nel prodotto finale:

Tabella 4: Problemi riscontrati durante il collaudo della pipeline e correzioni applicate

Problema riscontrato in fase di test	Causa	Correzione
Build HTML fallita in rete con restrizioni	L'@import di Google Fonts nel CSS blocca --self-contained se il font remoto non è raggiungibile	Rimosso l'@import esterno, sostituito con uno stack di font di sistema
Grafico con titolo tagliato ai bordi	Testo del titolo più largo della figura, nessun a-capo automatico	Aggiunto wrapping del testo del titolo nello script <code>make_chart.py</code>
Contenuto in HTML raw (badge, riquadri) scomparsa nel PDF	Pandoc scarta l'HTML grezzo per l'output LaTeX	Sostituito con <i>fenced div</i> Pandoc (<code>::: {.classe} ... :::</code>), compatibili con entrambi i formati
Capolettera sovrapposto alla prima riga di testo nel PDF WeasyPrint	<code>float</code> combinato con <code>::first-letter</code> è reso in modo diverso da WeasyPrint rispetto ai browser	Aggiunta una regola <code>@media print</code> che disattiva il <code>float</code> e usa una lettera più grande in linea solo in stampa

Tabella 4: Problemi riscontrati durante il collaudo della pipeline e correzioni applicate

Problema riscontrato in fase di test	Causa	Correzione
Grafici e riquadri non centrati nell'HTML	La shorthand <code>margin: Xrem 0</code> azzerava i margini laterali, sovrascrivendo <code>margin: 0 auto</code>	Corretto in <code>margin: Xrem auto</code> su <code>figure</code> , <code>.callout</code> e <code>hr</code>

Validazione degli output: ogni testo generato è stato revisionato manualmente per verificarne l'accuratezza rispetto agli abstract originali e la correttezza dei riferimenti bibliografici; ogni script e ogni modifica allo stile sono stati eseguiti e il risultato ispezionato visivamente (rendering di pagina di HTML e PDF) prima dell'inclusione nel prodotto finale.

Contributo e limiti:

- *Riduzione dei tempi:* significativa sia nella redazione dei contenuti sia nello sviluppo dell'automazione, grazie alla capacità di scrivere, eseguire e correggere codice nello stesso ambiente
- *Qualità:* il tono divulgativo è risultato coerente con il target; alcune formulazioni hanno richiesto piccole correzioni
- *Limiti:* il LLM non è in grado di verificare autonomamente la licenza degli articoli né di accedere a reti universitarie con le stesse restrizioni dell'utente finale; la ricerca automatica è stata validata solo strutturalmente e richiede una verifica in rete aperta alla prima esecuzione reale
- *Intervento umano necessario:* selezione degli articoli, verifica delle licenze, controllo dei dati numerici, revisione finale di testo, codice e stile

5 Valutazione dei risultati raggiunti

5.1 Valutazione del flusso di produzione

Criterio	Valutazione
Riduzione dei tempi	Significativa nella fase di redazione grazie all'uso del LLM; il flusso Pandoc automatizza la produzione di output multipli da unico sorgente, e la ricerca delle fonti non richiede più consultazione manuale
Riduzione degli errori	La separazione tra contenuto, metadati e stile riduce il rischio di inconsistenze tra i formati di output; l'uso di <i>fenced div</i> evita la perdita di contenuto tra HTML e PDF
Qualità dei documenti	Output HTML e PDF di qualità professionale, con identità visiva propria, indice automatico e grafici integrati
Raggiungimento nuovi canali	Il formato HTML self-contained è compatibile con newsletter, blog e CMS senza dipendenze esterne; il PDF con distribuzione formale

Criterio	Valutazione
Soddisfaccimento scenari d'uso	Le tre personas identificate trovano risposte adeguate nel prodotto; la persona Giulia in particolare beneficia dei grafici in apertura per un aggiornamento rapido

5.2 Confronto AS-IS / TO-BE

AS-IS (flusso tradizionale senza le tecnologie adottate): redazione manuale in Word, ricerca delle fonti interamente manuale su Google Scholar, esportazione PDF manuale, nessuna separazione tra contenuto e metadati, nessun elemento grafico a corredo del testo, aggiornamento costoso, nessuna automazione della conversione multiformato.

TO-BE (flusso adottato nel progetto): redazione in Markdown con metadati YAML separati, ricerca delle fonti assistita da uno script che interroga OpenAlex e restituisce risultati filtrati e ordinati, generazione dei contenuti assistita da LLM, due grafici generati automaticamente e integrati nel numero, conversione automatica in output multipli tramite Pandoc, versioning su GitHub, flusso completamente riproducibile tramite un unico script.

5.3 Limiti emersi

- Lo script di ricerca e la generazione live dei grafici richiedono accesso a `api.openalex.org`; in reti con restrizioni la modalità `--offline` garantisce comunque un output completo, a partire da dati reali già raccolti
- WeasyPrint supporta solo un sottoinsieme delle proprietà CSS3 disponibili nei browser (ad es. `box-shadow` e alcune sintassi di `@media` vengono ignorate silenziosamente): richiede quindi una verifica visiva del PDF dopo ogni modifica allo stile, non solo dell'HTML
- La selezione finale degli articoli tra i risultati restituiti dalla ricerca automatica resta manuale, per scelta metodologica coerente con la richiesta della traccia di dimostrare capacità di selezione editoriale
- Il flusso non include ancora un sistema di aggiornamento periodico schedato

6 Conclusioni

Il progetto ha dimostrato la fattibilità di un flusso di produzione editoriale digitale basato su tecnologie open source e standard aperti, con integrazione di strumenti di intelligenza artificiale generativa sia per la fase di redazione sia per lo sviluppo dell'automazione e dello stile. Gli obiettivi definiti dagli scenari d'uso sono pienamente raggiunti: il prodotto è accessibile, aggiornabile, distribuibile su canali multipli e corredato di fonti verificabili e di elementi visivi a supporto della lettura.

L'integrazione del LLM in modalità agentica ha permesso non solo di generare testo ma di sviluppare, eseguire e collaudare codice funzionante, individuando e correggendo in corso d'opera problemi che sarebbero emersi solo in fase di utilizzo reale (si veda la Tabella 4). Il flusso è documentato e riproducibile tramite il repository GitHub allegato.

I principali margini di miglioramento riguardano l'introduzione di un sistema di aggiornamento periodico schedato e l'estensione della ricerca automatica a un ventaglio più ampio di argomenti, oltre i due attualmente trattati.

Bibliografia e sitografia

1. P. Ceravolo, “Lezioni di editoria digitale,” 2025. Disponibile: <https://myariel.unimi.it/course/view.php?id=7524>
2. Faiyazuddin et al., “The Impact of Artificial Intelligence on Healthcare: A Comprehensive Review of Advancements in Diagnostics, Treatment, and Operational Efficiency,” *Health Science Reports*, 8: e70312, 2025. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11702416/>
3. Botha et al., “Artificial Intelligence in Healthcare: Perceived Threats to Patient Rights and Safety,” *Archives of Public Health*, 2024. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11515716/>
4. Adebesin et al., “The Role of Social Media in Health Misinformation and Disinformation During the COVID-19 Pandemic,” *JMIR Infodemiology*, 2023. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10551800/>
5. Yeung et al., “Medical and Health-Related Misinformation on Social Media: Bibliometric Study,” *Journal of Medical Internet Research*, 2022. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8793917/>
6. F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” *MIS Quarterly*, 1989.
7. Pandoc documentation. <https://pandoc.org/>
8. OpenAlex documentation. <https://docs.openalex.org/>
9. GitHub repository del progetto: <https://github.com/Elbara12/EditoriaDigitale>